

Codeforces Mashup Generator

Deep Dive: Exhaustive Technical Explainer

Prepared for Salman Adnan's portfolio

What this document is

This is a beginner-safe, exhaustive walkthrough of the `codeforces-mashup-generator` project: every file, every function, every design decision, and every technical term is explained the first time it appears. It is a working/study document, not part of the published portfolio website. Everything below is drawn directly from the project's source code (`generate_mashup.py`), `README.md`, `requirements.txt`, and `.env.example`. Where something is inferred rather than explicitly stated, it is marked in a **Flagged / Inferred** box.

Contents

1	What Problem Is This Solving?	3
1.1	Background: competitive programming and Codeforces	3
1.2	Who built it and how	3
2	Project Anatomy: What Files Exist and Why	4
3	High-Level Architecture	4
4	The CLI Layer: <code>parse_args</code>	5
5	Fetching Data From Codeforces	6
5.1	<code>fetch_problemset(lang, delay)</code>	6
5.2	<code>fetch_user_status(handle, delay)</code>	7
5.3	<code>collect_solved(handles, delay)</code>	8
6	Filtering: Turning Two Solved-Sets Into a Candidate List	9
6.1	<code>get_filtered_problems(args)</code>	9
6.2	<code>passes_tag_filter(problem)</code>	10
7	Selecting and Writing Output	10
7.1	<code>pick_choices_per_rating(filtered, n)</code>	10
7.2	<code>save_to_excel(choices, filename)</code>	10
8	Error-Handling Philosophy	11
9	Dependencies	11

10 Verified Example Run	11
11 Known Limitations (author's own assessment)	12
12 Glossary	12

1 What Problem Is This Solving?

1.1 Background: competitive programming and Codeforces

Term: Competitive programming

A discipline where programmers solve short, well-defined algorithmic puzzles under a time limit, usually during a live online contest. Problems are scored by difficulty, and solving them correctly (an “Accepted” verdict) earns points.

Term: Codeforces

A website (<https://codeforces.com>) that hosts competitive-programming contests. It maintains a public problem archive and a numeric *rating* for every problem (roughly: how hard it is) and for every user (roughly: how skilled they are). It also exposes a public, free, read-only **API** (see below) that anyone can query to fetch problem lists and user submission histories as structured data.

Term: Mashup contest

An informal, self-organised contest where a group picks a handful of problems (often one per difficulty level) from the existing archive and treats them as a fresh contest to practice together. Unlike an official contest, the organiser must manually pick problems, which creates the exact fairness problem this project solves.

The README states the motivating problem directly: “When our study group ran mashup contests, someone always got a problem they’d already solved, which ruins the point.” If a problem is handed to someone who has already solved it before, that problem no longer tests anything for them — they already know the answer. The project’s core job is to automatically pick a batch of Codeforces problems, at chosen difficulty levels, that no one in the group has solved yet, while optionally also requiring that the problem is known to be solvable (some strong reference solver has already solved it, as a rough “this problem is not broken/mislabeled” quality signal).

1.2 Who built it and how

This is solo work by Salman Adnan (stated directly in the README). It is implemented as a single Python script, `generate_mashup.py` (229 lines), with no other application code. The README additionally documents that the script began as a hardcoded configuration block (handle lists and settings baked directly into the file, evidenced by the leftover file `all_users.txt` described in §2) and was later refactored to accept command-line arguments via Python’s `argparse` library.

Flagged / Inferred

Inferred, not directly stated: the exact date of the original hardcoded version versus the `argparse` refactor is not given anywhere in the repository; only the fact that a refactor happened (per the README’s “Challenges” and “Usage” sections) is confirmed.

2 Project Anatomy: What Files Exist and Why

File	Role
<code>generate_mashup.py</code>	The entire application: CLI parsing, API calls, filtering logic, Excel output.
<code>README.md</code>	Project description, usage examples, and the author's own account of challenges and lessons learned.
<code>requirements.txt</code>	Pinned Python dependency ranges: <code>requests</code> , <code>pandas</code> , <code>openpyxl</code> .
<code>.env.example</code>	Documents that <i>no</i> environment variables are used — the Codeforces API needs no key and all configuration is via CLI flags.
<code>all_users.txt</code>	A leftover flat text file containing what looks like an earlier, hardcoded version of the <code>solver_handles</code> / <code>exclude_handles</code> Python lists (with a <code># === CONFIGURATION ===</code> comment header), i.e. a pre-refactor snapshot of the same data that now lives as <code>DEFAULT_INCLUDE_USERS</code> / <code>DEFAULT_EXCLUDE_USERS</code> inside the script. It is not imported or read by <code>generate_mashup.py</code> at runtime.
<code>LICENSE</code>	MIT License, copyright Salman Adnan, 2026.
<code>.gitignore</code>	Standard Python/venv/OS ignore patterns (<code>__pycache__</code> / <code>.venv</code> / <code>.env</code> , log files, etc.).

Flagged / Inferred

Flagged: `all_users.txt` is not referenced anywhere in `generate_mashup.py`. Its content (a Python-syntax `solver_handles = [...] / exclude_handles = [...]` block) strongly suggests it is a stray leftover from the pre-argparse hardcoded version described in the README, kept in the repo but functionally inert. This is an inference from file content, not a statement in the README.

3 High-Level Architecture

Term: CLI (command-line interface)

A program controlled by typing a command and flags into a terminal (e.g. `python generate_mashup.py --ratings 800 1200`) rather than through a graphical window. Flags customise behaviour without editing the source code.

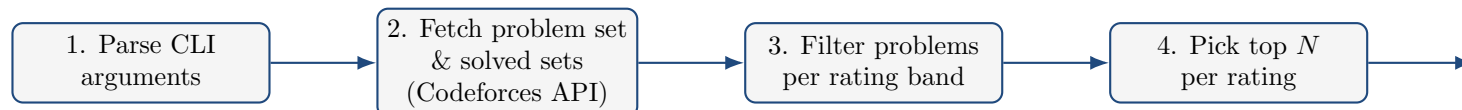
Term: API (Application Programming Interface) / REST API

A way for one program to ask another program (usually running on a remote server) for data over the internet, using standard web requests (HTTP). A *REST* API structures those requests as URLs, and typically returns data as **JSON** (see below). Here, the remote server is Codeforces itself.

Term: JSON (JavaScript Object Notation)

A plain-text data format for representing nested lists and key/value records, e.g. `{"status": "OK", "result": [...]}`. Almost every modern web API, including Codeforces's, replies in JSON, and Python's `requests` library can parse it directly into Python dictionaries and lists with `.json()`.

The script has exactly one entry point (the `if __name__ == '__main__':` block) and five conceptual stages, run strictly in sequence:



Each stage maps directly onto one or more functions in the source file, listed here and explained in full in §??:

Stage	Function(s)	Section
1	<code>parse_args</code>	4
2	<code>fetch_problemset</code> , <code>fetch_user_status</code> , <code>collect_solved</code>	5.1, 5.2, 5.3
3	<code>get_filtered_problems</code> , <code>passes_tag_filter</code>	6.1, 6.2
4	<code>pick_choices_per_rating</code>	7.1
5	<code>save_to_excel</code>	7.2

4 The CLI Layer: `parse_args`

Term: `argparse`

Python's standard-library module for turning command-line flags (like `--ratings 800 900`) into a Python object with named attributes (`args.ratings`). It also auto-generates a `--help` message.

`parse_args()` builds an `ArgumentParser` and defines every flag the script accepts. `formatter_class=argparse.ArgumentDefaultsHelpFormatter` means `--help` automatically prints each flag's default value alongside its description.

Flag	Default	Meaning
<code>--include-users</code>	46 named handles	Problems must be solved by at least one of these Codeforces handles (a rough "this problem is legitimate/solvable" quality signal).
<code>--exclude-users</code>	34 named handles	Problems must <i>not</i> be solved by any of these handles (the group members the mashup is being built for).
<code>--no-include-filter</code>	off	Skip the include-users requirement entirely.
<code>--no-exclude-filter</code>	off	Skip the exclude-users requirement entirely.
<code>--ratings</code>	800–2000 step 100 (13 values)	Which Codeforces difficulty ratings to fetch problems for.

Flag	Default	Meaning
<code>--choices-per-rating</code>	<code>10</code>	How many surviving problems to keep per rating band.
<code>--include-tags</code>	<code>(empty)</code>	Codeforces problem tags (topics, e.g. <code>greedy</code> , <code>dp</code>) to require, per <code>--tag-mode</code> .
<code>--exclude-tags</code>	<code>(empty)</code>	Tags that disqualify a problem outright, always applied.
<code>--tag-mode</code>	<code>all</code>	How <code>--include-tags</code> is matched: <code>some</code> (at least one), <code>all</code> (every listed tag present), <code>strict</code> (tag set matches exactly), <code>off</code> (ignore include tags).
<code>--lang</code>	<code>en</code>	Statement language passed straight to the Codeforces API's own <code>lang</code> parameter.
<code>--delay</code>	<code>0.5 (seconds)</code>	Pause between successive API calls, to respect Codeforces's rate limit.
<code>-o / --output</code>	<code>filtered_problems.xlsx</code>	Output Excel filename.

The default include/exclude lists (`DEFAULT_INCLUDE_USERS`, `DEFAULT_EXCLUDE_USERS`, defined at the top of the file) reproduce, in the author's words, "the original study-group configuration": 46 trusted competitive programmers as the include list (well-known names such as `tourist`, `jiangly`, `ecnerwala`), and 34 group-member handles as the exclude list.

5 Fetching Data From Codeforces

5.1 `fetch_problemset(lang, delay)`

Term: Endpoint

A specific URL on an API that returns one particular kind of data. Codeforces exposes several; this project calls exactly two: `problemset.problems` and `user.status`.

Calls `https://codeforces.com/api/problemset.problems?lang=<lang>`, which returns *every* problem in the Codeforces archive as one flat JSON list, each with fields such as `contestId`, `index` (e.g. "A", "B2"), `rating`, `name`, and `tags`. A 60-second `timeout` guards against a hung connection, and `r.raise_for_status()` raises an exception if the HTTP response itself signals failure (e.g. a 404 or 500 status code).

```
def fetch_problemset(lang, delay):
    url = f"https://codeforces.com/api/problemset.problems?lang={lang}"
    r = requests.get(url, timeout=60)
    r.raise_for_status()
    data = r.json()
    if data.get('status') == 'OK':
        time.sleep(delay)
        return data['result'].get('problems', [])
    ...
```

Term: Two-layer error checking

Codeforces (like many APIs) can fail in two different ways: the HTTP transport itself can fail (network down, server error — caught by `raise_for_status()`), *or* the HTTP request can succeed (status code 200 "OK") while the JSON body itself says `{"status": "FAILED", ...}` (e.g. a bad handle). The code checks `data.get('status') == 'OK'` separately for exactly this reason — a lesson the README calls out explicitly under “What I learned”.

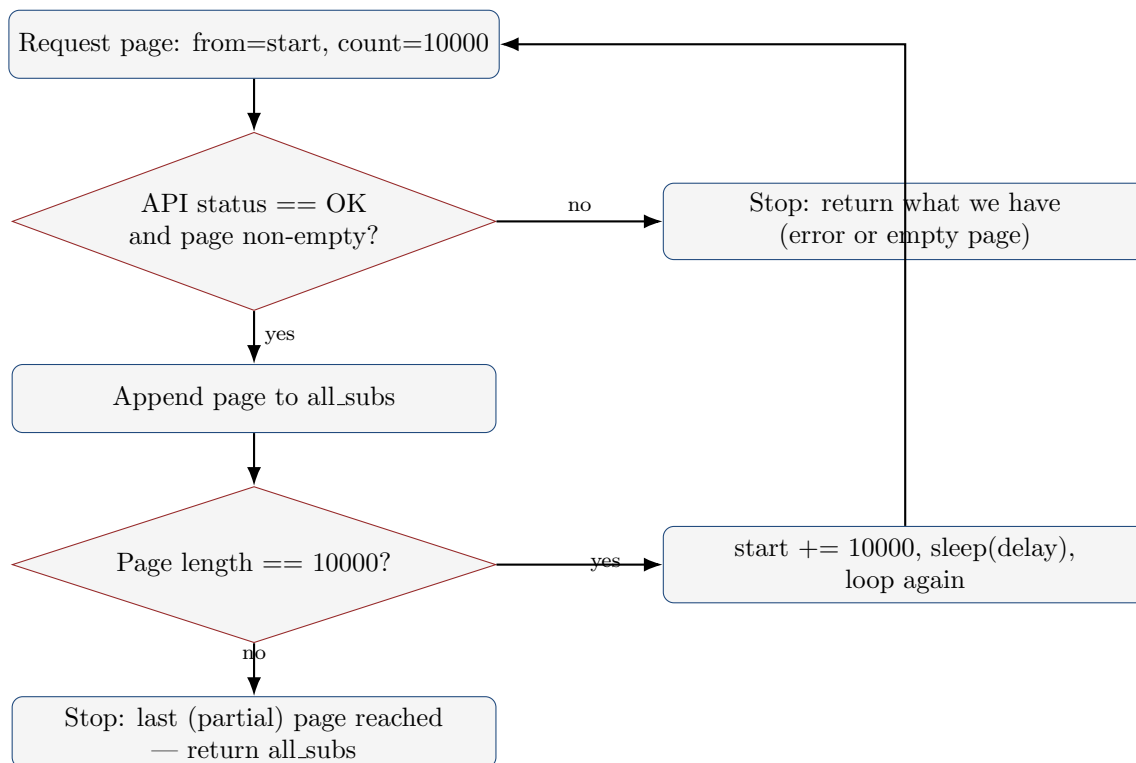
5.2 `fetch_user_status(handle, delay)`

Term: Pagination

When an API has too much data to return in one response, it returns it in “pages” and lets the caller ask for the next page with parameters like `from` (starting index) and `count` (page size). The caller must keep requesting pages until a stop condition is met.

This function downloads *every submission a single Codeforces user has ever made*, because `user.status` can return tens of thousands of rows for prolific users and Codeforces cannot hand them all back in one response. It loops, requesting `count=10000` submissions starting at `from=start`, and:

- stops the loop the moment a page comes back with *fewer* rows than the requested batch size (10,000) — that means it was the last page;
- stops (with an empty list) if the API reports a non-OK status, e.g. an unknown handle, printing the API’s own error comment;
- catches any exception (network error, timeout, malformed JSON) per handle, so one bad handle cannot crash the whole run;
- sleeps `delay` seconds after every successful page fetch to stay under Codeforces’s rate limit.



5.3 `collect_solved(handles, delay)`

Term: Set (data structure)

A Python collection that holds unique items with no duplicates and supports very fast “is this item in the set?” checks (close to constant time, i.e. the check takes roughly the same time no matter how large the set is).

Term: Tuple

A small, fixed, ordered, immutable (unchangeable) group of values, written like `(cid, idx)`. Because tuples of plain values are hashable, they can be stored directly inside a Python `set`.

For a list of handles, this calls `fetch_user_status` on each one, and for every submission with `verdict == 'OK'` (Codeforces’s code for “accepted, fully correct”), it records the problem’s identity as a `(contestId, index)` tuple in a running `set`. It is called twice by `get_filtered_problems`: once for `include_users` (producing `include_solved`) and once for `exclude_users` (producing `exclude_solved`). Using the exact same helper for both, rather than duplicating the loop, is a refactor the README explicitly credits as a bug-avoidance move: “a fix to the verdict check had to be made in two places” before the helper existed.

A subtlety documented in the README and visible in the code:

```

prob = s.get('problem', {})
cid = prob.get('contestId') or s.get('contestId')
idx = prob.get('index')
if cid and idx:
    solved.add((cid, idx))

```

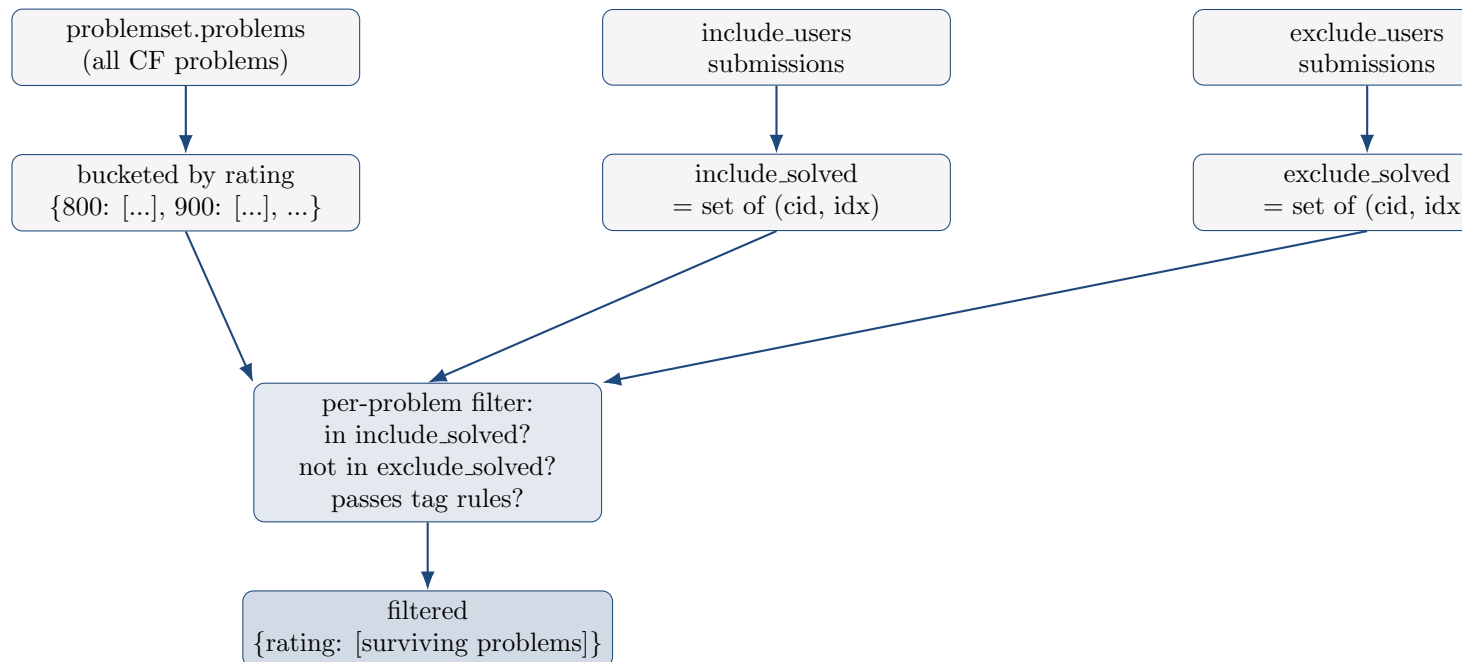
Some submission records do not carry `contestId` on the nested `problem` object; it can instead live one level up, directly on the submission. The `or` fallback handles both shapes, and the record is only added to the set once *both* `cid` and `idx` are confirmed present — otherwise a handful of malformed or gym-contest entries would pollute the set with incomplete keys that could never match a real problem.

6 Filtering: Turning Two Solved-Sets Into a Candidate List

6.1 `get_filtered_problems(args)`

This is the orchestration function for stage 3. In order:

1. Calls `fetch_problemset` once, then buckets every returned problem into a dictionary `by_rating: {rating: [problem, ...]}`, keeping only problems whose `rating` field is one of the requested `--ratings` values. (Problems with no rating, or a rating outside the requested list, are silently dropped here.)
2. Builds `include_solved` via `collect_solved(args.include_users, ...)` — unless `--no-include-filter` was passed, in which case it stays an empty set and is never consulted.
3. Builds `exclude_solved` the same way from `args.exclude_users`.
4. For each rating bucket, keeps only problems where *all* of the following hold:
 - include filter is off, **or** the problem's key is in `include_solved`;
 - exclude filter is off, **or** the problem's key is *not* in `exclude_solved`;
 - `passes_tag_filter(p)` returns `True` (§6.2).



6.2 `passes_tag_filter(problem)`

Term: Codeforces tags

Short topic labels Codeforces attaches to each problem, such as **greedy**, **dp** (dynamic programming), **graphs**, or **bitmasks**, describing the technique(s) needed to solve it.

This nested function (defined inside `get_filtered_problems` so it can see `args` without extra parameters) first always drops a problem if any of its tags appear in `--exclude-tags` (this check is unconditional). Then, only if `--include-tags` is non-empty *and* `--tag-mode` is not off, it applies one of three set-comparison rules:

Mode	Set operation	Meaning
some	<code>tags & inc_set</code> non-empty	At least one requested tag is present.
all	<code>inc_set.issubset(tags)</code>	Every requested tag is present (problem may have others too).
strict	<code>tags == inc_set</code>	The problem's tag set is <i>exactly</i> the requested set, no more, no fewer.

`tag-mode all` is the script's own default. Note this is a change from the original hardcoded version: the README states the pre-refactor script had `include_tags = ["bitmasks"]` hardcoded as a leftover from its last real use, and the author deliberately dropped that as a default when moving to the CLI, since a session-specific tag was not a sensible general default.

7 Selecting and Writing Output

7.1 `pick_choices_per_rating(filtered, n)`

Trivial by design: for each rating, takes `plist[:n]` — a Python slice meaning “the first `n` items of the list, in whatever order they already came in.” Because the problems arrive from `problemset.problems` in (observed) roughly-newest-first API order, this means the chosen problems are the newest survivors, not a random sample. The README lists this as a known limitation (§11).

7.2 `save_to_excel(choices, filename)`

Term: pandas DataFrame

A table-like Python data structure (rows and named columns) from the **pandas** library, commonly used for tabular data manipulation and export.

Term: openpyxl

A Python library that **pandas** uses under the hood to actually write **.xlsx** (Excel) files. It is listed in `requirements.txt` but never imported directly in the script — `df.to_excel(...)` calls it internally.

Flattens the `{rating: [problems]}` selection into one row per problem (fields: `rating`, `contestId`, `index`, `name`, comma-joined `tags`, and a direct clickable link built as `https://codeforces.com/problemset/probl`

builds a single `pandas.DataFrame`, and writes it with `df.to_excel(filename, index=False)`. If there are zero surviving rows across every rating, it prints “No problems to save.” and returns without writing a file at all.

8 Error-Handling Philosophy

Every network call in the script (`fetch_problemset`, and the loop inside `fetch_user_status`) is wrapped in a `try/except Exception` that prints a message and continues with an empty or partial result, rather than crashing the whole run. Combined with the JSON-level `status == 'OK'` check described in §5.1, this means:

- a single bad or renamed handle in a 40-plus-name list does not stop the run;
- a transient network hiccup on one page of one handle’s submissions truncates that handle’s solved-set rather than aborting;
- the script always exits with code 0 (success) even when it silently produced an empty or partial result — flagged explicitly by the author as a weakness under “What I’d do differently.”

Flagged / Inferred

The author’s own README flags a fairness implication of this design: if `user.status` fails for one of the *exclude* handles, that member’s solved problems are treated as *unsolved*, silently weakening the guarantee the whole tool exists to provide. The README proposes this specific failure mode should be a hard error instead of a warning. This is the author’s own assessment, not this document’s inference.

9 Dependencies

Package	Pinned range	Role
<code>requests</code>	<code>>=2.31,<3</code>	HTTP client used for every Codeforces API call.
<code>pandas</code>	<code>>=2.0,<3</code>	Builds the tabular <code>DataFrame</code> of selected problems.
<code>openpyxl</code>	<code>>=3.1,<4</code>	Excel-file writer engine used internally by <code>pandas</code> ’ <code>.to_excel()</code> .

No API key or environment variable is required anywhere (confirmed by `.env.example`): the Codeforces API is fully public, and all behaviour is controlled through CLI flags.

10 Verified Example Run

The README documents a small run the author verified against the live API (about 14 seconds):

```
python generate_mashup.py \
  --include-users ksun48 \
  --exclude-users salmanadnan2025 Musab1Blaser \
  --ratings 800 1200 1600 \
  --choices-per-rating 3 \
  --include-tags greedy --tag-mode some \
  -o mashup_test.xlsx
```

Reported result: 9 rows (3 per rating $\times$ 3 ratings), every row tagged **greedy**, none solved by the two excluded members, all solved by **ksun48**. The README also notes that during this run two stale handles in the default exclude list returned HTTP 400 and the script still completed and produced a correct sheet — a live confirmation of the per-handle error tolerance described in §6.

11 Known Limitations (author’s own assessment)

Directly from the README’s “What I’d do differently” section, not this document’s own judgment:

- Downloading *every* submission of every trusted include-handle is disproportionate to the question being asked (has this problem been solved, yes/no). Handles like **tourist** have tens of thousands of submissions, so a default run transfers hundreds of megabytes to answer a set-membership question. Caching solved-sets to disk, or using the **solvedCount** field already present on **problemset.problems** entries, would cut runtime substantially.
- Selection (`plist[:n]`) always takes the newest problems in API order, not a random sample, so repeat runs with identical filters return near-identical sheets. A seeded `random.sample` would fix this.
- A failed `user.status` call for an *excluded* member silently weakens the exclude guarantee (see notebbox above); the author states this should be a hard error, not a warning.
- All diagnostics go to `stdout` via `print`, and the exit code is always 0; proper logging and a nonzero exit code on an empty/failed result would make the script safely scriptable in a pipeline.
- `--choices-per-rating` is one global number; the pre-refactor version allowed a per-rating count via a dict, which the author dropped as unused, but notes a `RATING=N` style flag could restore that flexibility.

12 Glossary

Term	Definition
API	A way for programs to request data from another program/server over the internet.
REST API	An API whose requests are structured as URLs and whose replies are typically JSON.
JSON	A plain-text format for nested key/value data.
Endpoint	One specific URL/operation exposed by an API.
Pagination	Splitting a large API result across multiple requested “pages.”
CLI	A program controlled by typed commands/flags rather than a graphical window.
argparse	Python’s standard library for parsing CLI flags into a Python object.
Set (Python)	An unordered collection of unique, hashable items with fast membership checks.
Tuple	A fixed-size, ordered, immutable group of values, e.g. (<code>contestId</code> , <code>index</code>).
DataFrame	A pandas table structure of rows and named columns.

Term	Definition
Rating (Codeforces)	A numeric difficulty score Codeforces assigns to problems (and skill score for users).
Tag (Codeforces)	A topic label on a problem (e.g. <code>greedy</code> , <code>dp</code>).
Verdict	Codeforces’s judgment of a single submission; OK means fully correct (“Accepted”).
Rate limiting	A server-imposed cap on how often a client may call an API, enforced here voluntarily via <code>--delay</code> sleeps.
Timeout (HTTP)	A maximum wait time for a network response before giving up, preventing a hung connection from stalling the whole script.
